# Técnicas de los Sistemas Inteligentes

## Curso Académico 2025-26

## Práctica 1: Desarrollo de agentes basado en técnicas de búsqueda heurística dentro del entorno GVGAI

La Práctica 1 consiste en desarrollar cuatro agentes basados en técnicas de búsqueda diferentes (incluyendo técnicas de búsqueda no informada y de búsqueda heurística, tanto offline como en tiempo real) dentro del entorno GVGAI[1]. Estos agentes deberán guiar a un avatar para resolver un mismo juego en distintos niveles. El juego escogido es una modificación de *Catapults*, disponible con el índice 122 (la versión original corresponde al índice 19) dentro de los juegos de tipo "*singleplayer*" recogidos en el fichero "*all_games_sp.csv*" de GVGAI. Para utilizar esta nueva versión del juego, deberán incorporarse todos los cambios indicados en las diapositivas de presentación de la práctica.

### 1. Descripción general del juego

*Catapults (modificado)* es un videojuego sencillo cuyo objetivo consiste en alcanzar el portal de salida recorriendo un mundo cuadriculado y laberíntico en donde el objetivo es salir por una puerta que inicialmente está cerrada. Para lograr el objetivo, el agente debe evitar los obstáculos (plantas y agua) y recoger una llave antes de llegar a la puerta para poder abrirla y salir.

Para evitar las zonas acuáticas, el mapa dispone de diversas catapultas, simbolizadas mediante flechas direccionales. Al entrar en una catapulta, el agente se transforma en murciélago y avanza automáticamente en la dirección indicada hasta colisionar con un obstáculo. Tras la colisión, el avatar se sitúa en la casilla inmediatamente anterior y recupera su forma original, que denominaremos "vampiro". El uso de una catapulta al entrar andando en ella consume 1 moneda. Si el agente atraviesa otra catapulta mientras está volando, no se aplica un coste adicional, aunque sí se actualiza la dirección de vuelo. En todos los casos, la catapulta utilizada desaparece y la casilla pasa a contener césped transitable. Si el agente intenta usar una catapulta sin disponer de monedas, el jugador muere y la partida termina.

El agente comienza el juego con el inventario vacío (es decir, sin monedas ni llave) y, si necesita hacer uso de las catapultas, deberá recoger alguna de las monedas disponibles en el mapa. Las monedas (de las cuales el agente puede tener hasta 5 en su inventario[2]) pueden encontrarse tanto sobre suelo transitable como encima del agua, donde solo

[1] Se puede descargar desde https://github.com/GAIGResearch/GVGAI/archive/master.zip

[2] Y, a partir de ese valor, el avatar, sencillamente, puede transitar por las casillas con monedas, pero sin recoger ninguna (hasta que el número de monedas en el inventario vuelva a ser menor de 5).

podrían ser recogidas mientras vuela. Siempre habrá al menos una llave en el mapa y se encontrará sobre el césped. Se garantiza que todos los mapas serán siempre resolubles, es decir, siempre habrá al menos un camino que lleve a la meta (siempre y cuando el algoritmo de búsqueda garantice completitud) y también se garantiza que, en los bordes del mapa, siempre habrá elementos de tipo planta/muro o meta (la meta, en concreto, puede tanto estar en el borde como en cualquier otra posición del mapa).

**Acciones**: se pueden ejecutar 4 acciones de movimiento (IZQUIERDA, DERECHA, ARRIBA, ABAJO). También se puede realizar la acción NIL (acción nula) que, en realidad, consiste en no hacer nada y que el avatar, como consecuencia, no realice ningún movimiento adicional a la lógica del juego. La acción NIL deberá ser utilizada durante todo el proceso de uso de una catapulta, ya que durante ese tiempo ningún otro comando de movimiento tendrá efecto y se dependerá exclusivamente de la lógica del juego. El proceso de uso de la catapulta implica una acción de tipo NIL para cada transformación (de vampiro a murciélago y de murciélago a vampiro), una acción de tipo NIL adicional al sobrepasar una nueva catapulta, y una acción de tipo NIL por cada casilla que se avance.

*Figura 1: Mapa del primer nivel (levelIdx=0) de Catapults modificado (gameIdx = 122) en el framework GVGAI (General Video Game AI). Podemos ver el avatar en la esquina superior izquierda, la puerta de salida en la esquina inferior derecha, múltiples catapultas direccionales (que permiten traspasar los obstáculos acuáticos), muros de colisión en forma de planta, y la llave para abrir la puerta tres casillas a la izquierda de la puerta de salida.*

## 2. Descripción de la tarea a realizar

El objetivo de la práctica es que los estudiantes se familiaricen con el análisis e implementación de técnicas de búsqueda. Para ello, el alumnado deberá implementar los siguientes algoritmos de búsqueda:

- **Búsqueda no informada** (offline) con **Búsqueda en profundidad**
- **Búsqueda heurística** (offline) con **A***
- **Búsqueda heurística** (online) **en tiempo real** con **RTA***
- **Búsqueda heurística** (online) **en tiempo real** con **LRTA* (k) con k=5**

Cada uno de estos algoritmos será ejecutado en mapas de diversa índole (pequeños, medianos, grandes, con más o menos catapultas, etc.). El objetivo es verificar que las implementaciones realizadas respetan la definición del juego propuesto y generalizan y escalan a diversas configuraciones del juego.

*Figura 2: Ejemplo de mapa de tamaño pequeño (denominado catapults2026_lvl4.txt en los mapas proporcionados a través de PRADO)*

A modo de ejemplo, se proporcionan mapas como los que se muestran en las Figuras 1, 2 y 3. No obstante, es importante remarcar que **los mapas de evaluación serán diferentes a los proporcionados durante la fase de implementación**. Es decir, es fundamental que el alumnado desarrolle mapas adicionales que verifiquen que se respetan todas las restricciones propuestas en la descripción y en la sección 3. Debemos recordar que las soluciones propuestas por los estudiantes deben ser generalistas, es decir, dentro de la tarea en cuestión, deben permitir resolver correctamente otros mapas de dificultad aproximadamente similar.

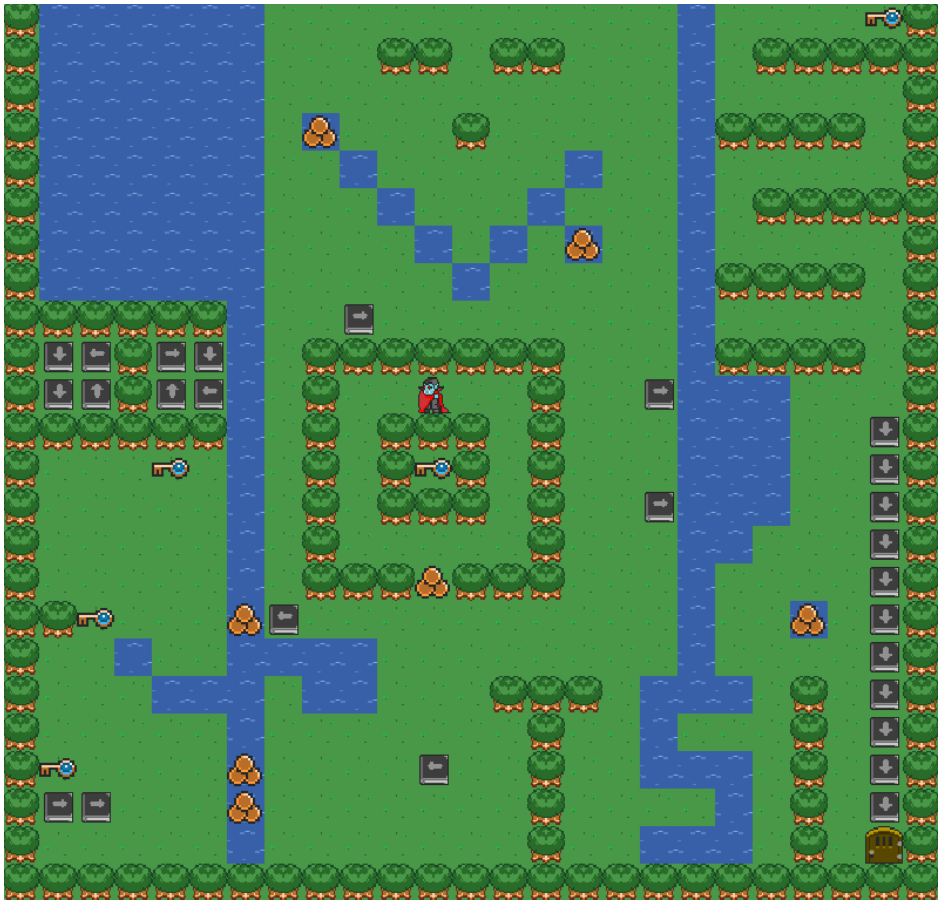

*Figura 3: Ejemplo de mapa de tamaño mediano (denominado catapults2026_lvl6.txt en los materiales proporcionados a través de PRADO)*

Como referencia para el alumnado, se proporciona el siguiente mapa resuelto (correspondiente con la Figura 4).

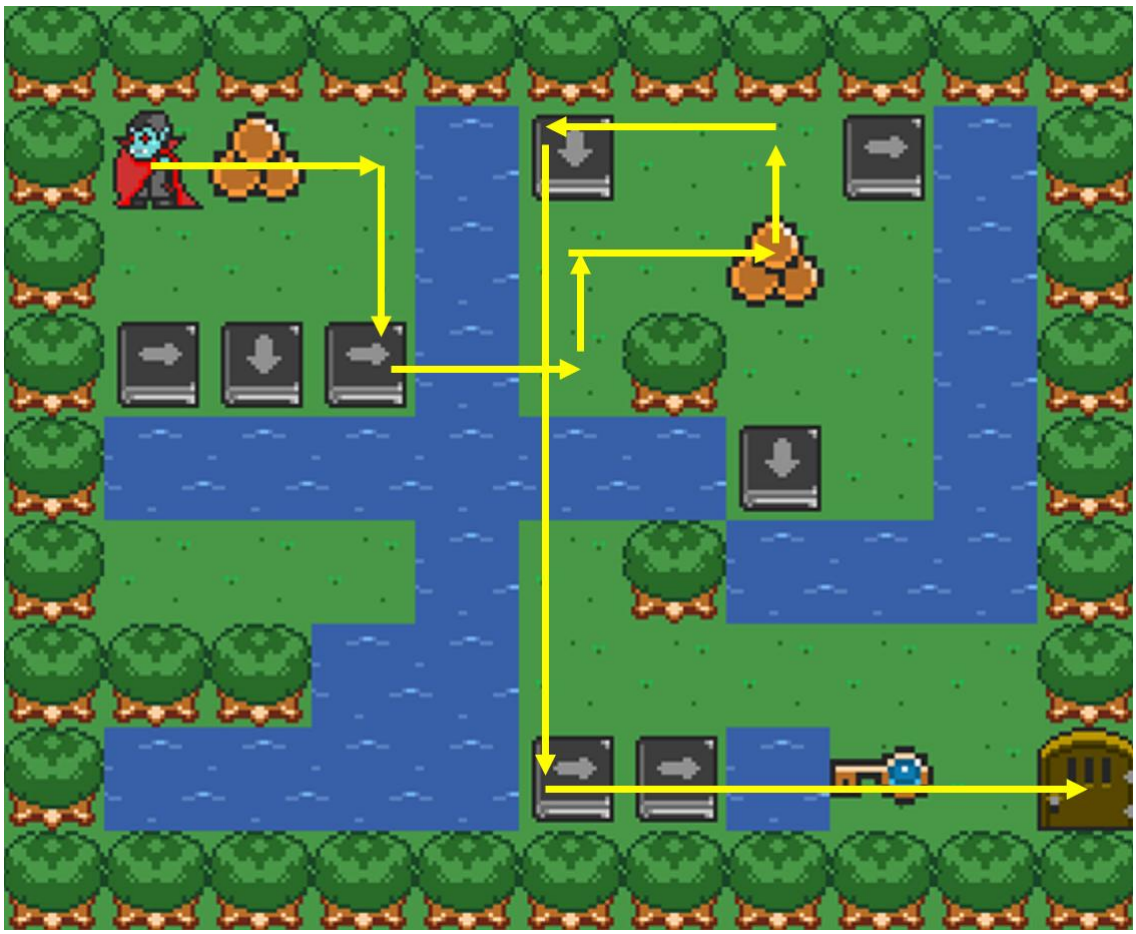

Figura 4: Ejemplo de mapa resuelto para facilitar la comprensión del juego por parte del alumnado. Se marca con flechas amarillas la trayectoria seguida por el avatar

La solución óptima de dicho mapa se corresponde con el siguiente plan:

ACTION_RIGHT
ACTION_RIGHT
ACTION_DOWN
ACTION_DOWN
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_UP
ACTION_RIGHT
ACTION_RIGHT
ACTION_UP
ACTION_LEFT
ACTION_LEFT

ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL
ACTION_NIL

Es decir, el avatar parte de la esquina superior izquierda (en modo “vampiro”), se desplaza a la derecha para coger la moneda (ACTION_RIGHT), de nuevo a la derecha (ACTION_RIGHT), abajo (ACTION_DOWN) y de nuevo abajo (ACTION_DOWN) para subirse a la catapulta. En la catapulta se transforma en murciélago (ACTION_NIL), y se desplaza hacia la derecha dos casillas (dos ACTION_NIL consecutivos) hasta chocar con un muro/árbol, que hace que se transforme de nuevo en vampiro, consumiendo un ACTION_NIL. A continuación, el avatar sube una casilla, y se mueve dos a la derecha para coger el dinero (ACTION_UP, ACTION_RIGHT, ACTION_RIGHT). Ahora, siguiendo las restricciones de expansión de vecinos, subirá y se desplazará dos casillas a la izquierda para subirse a la catapulta (ACTION_UP, ACTION_LEFT, ACTION_LEFT). Al subirse a la catapulta se realiza la transformación en murciélago (ACTION_NIL), y comienza el viaje hacia abajo hasta llegar a la siguiente catapulta, que cambia la dirección de desplazamiento. En ese proceso se invierten 6 ACTION_NIL. Al llegar a la catapulta inferior, se consume otro ACTION_NIL, se desplaza a la derecha (otro ACTION_NIL), se llega a otra catapulta (que consume un ACTION_NIL más), y habría otros 4 ACTION_NIL más que nos permitirían sobrevolar una casilla con agua, recoger la llave y atravesar el portal. En total, 28 acciones, que se corresponden con el camino óptimo para este mapa.

.

## 3. Implementación y evaluación de los algoritmos.

Toda la implementación debe desarrollarse dentro del siguiente **paquete Java** (que cada estudiante deberá crear al inicio de la práctica):

src/tracks/singlePlayer/evaluacion/src_APELLIDO1_APELLIDO2_NOMBRE

donde "nombre" y "apellidos" irán en **mayúscula** y sin guiones ni espacios (Java es case-sensitive). Dentro de este paquete, se definirán, al menos, las siguientes **clases Java** (case-sensitive):

- AgenteProfundidad
- AgenteAStar
- AgenteRTAStar
- AgenteLRTAStarK (k=5, debe poderse cambiar el valor de k en la implementación y hacer uso del soporte).

Se recomienda, para evitar problemas con la nomenclatura, seguir las instrucciones de la presentación de la práctica y hacer uso de la plantilla proporcionada.

Cada uno de los algoritmos evaluados será evaluado sobre 7 mapas de evaluación, que serán diferentes a los proporcionados al alumnado para la fase de implementación.

Previo al proceso de calificación, todas las prácticas se someterán a un sistema de detección de copias. También se revisará que los agentes de búsqueda en tiempo real no calculan el plan entero directamente, sino que operan acción a acción, como es natural y propio de este tipo de algoritmos. No cumplir con este criterio de implementación implicará que el agente correspondiente no recibe puntuación.

Todos los mapas contribuirán de forma uniforme a la calificación de los algoritmos siendo evaluados de la siguiente manera:

- **80% de la puntuación** se corresponderá al camino (path) encontrado, que será verificado acción a acción. La presencia de acciones nulas que no aportan nada (por ejemplo, al principio) o de acciones distintas a la nula durante el uso de una catapulta harán que, en lugar de recibir un 80% de la nota total del mapa, la puntuación se corresponda con el 40% (siempre y cuando se siga encontrando el camino esperado).
- El **20% restante** de la calificación de un mapa se corresponderá a las métricas complementarias y solo se evalúa si se ha obtenido puntuación en el apartado previo. Estas deben ser proporcionadas a través de los métodos de la clase planteada para ello en la plantilla (**MetricsProvider.java**), que no puede ser modificada bajo ningún concepto. Se deben proporcionar las siguientes métricas por algoritmo:

- **Profundidad**: N.º de acciones del plan (5%), nodos expandidos (10%), profundidad máxima evaluada (5%)
- **A***: N.º de acciones del plan (5%), nodos en abiertos (5%) y cerrados (5%) al final de la ejecución, nodos expandidos (5%).
- **RTA*:** N.º de acciones del plan (10%), nodos expandidos (10%).
- **LRTA*(k):** N.º de acciones del plan (5%), nodos expandidos (5%), número de actualizaciones de la heurística (10%).

Se recuerda al alumnado que la ejecución de todos estos algoritmos es determinista, es decir, todos los alumnos deberían obtener el mismo camino si se respetan todas las restricciones propuestas. En ocasiones, es posible que aparezcan nuevas soluciones en base a otras representaciones del espacio de estados que no haya tenido en cuenta el profesorado de la asignatura. En ese caso, dicha solución alternativa se considerará válida si se respetan todas las restricciones propuestas a continuación, aunque es recomendable que se lo comunique al profesorado durante la fase de implementación.

**Las siguientes restricciones son de obligatorio cumplimiento y pueden invalidar parcial o totalmente la puntuación del agente si no se respetan:**

- **Restricción de nomenclatura**. La nomenclatura anterior (nombre de paquete y clases) debe ser respetada **estrictamente**, ya que la corrección de esta parte se hará de forma estricta y automática. Se recomienda encarecidamente que sigan las instrucciones de la presentación y la plantilla inicial

- **Restricciones en la ejecución.** Es importante tener en cuenta que cada ciclo de decisión está limitado de tal forma que la decisión de qué acción ejecutar en **cada *tick* debe tomarse en un tiempo no superior a 40ms**[3]. Además, **la ejecución se debe limitar a un máximo de 5.000 *ticks***. Por tanto, es importante que la implementación sea suficientemente eficiente, con una estructura de datos ligera y ágil en las operaciones requeridas, y que la definición del juego y de GVGAI permita la ejecución hasta 5.000 *ticks* (ver la presentación de la práctica para más detalles). Es decir, no debe depender de los métodos del motor (que son más lentos) sino que debe hacer su propia implementación del estado y los nodos de búsqueda, así como de los algoritmos planteados.

- **Restricciones en el constructor de los agentes**. Las operaciones realizadas en el constructor de los agentes se limitarán a acciones "menores" (de inicialización). Es decir, en el constructor puede generar las estructuras de datos estáticas y dinámicas y hasta el nodo inicial de búsqueda. **Toda acción relativa al método de búsqueda deberá realizarse dentro del método "act"**. Para ello, todos los agentes de búsqueda offline calcularán la ruta completa desde la posición actual del avatar hasta la casilla objetivo en la primera ejecución del método "act", eligiendo como primera acción la primera casilla a la que

[3] Si la acción seleccionada en el método *act* se decide entre 40-50ms GVGAI devuelve la acción nula; si el tiempo es mayor a 50ms se descalifica al agente.

desplazarse, y en las llamadas sucesivas a este método simplemente seguirán ejecutando los movimientos para desplazarse por la ruta ya calculada.

- **Restricciones de la heurística**. En los algoritmos heurísticos (A*, RTA* y LRTA*(k)), la función heurística será siempre la **distancia Manhattan** desde el estado actual hasta el portal de salida.

- **Restricción de representación del estado.** La representación del estado debe ser la misma para todos los algoritmos de búsqueda, independientemente de si son offline u online, y dicha representación debe ser completa y mantener toda la información necesaria para representar todos los elementos dinámicos.

- **Restricciones de expansión de vecinos**. Para la expansión de un nodo, el **orden de expansión** de sus **vecinos** será siempre el mismo: **Derecha, Arriba, Izquierda, Abajo**. Además, en caso de requerir un criterio adicional para ordenar los nodos expandidos, se usará un criterio histórico a modo de cola FIFO. En el caso de A*, a mismo f(n), el desempate inicial se hará según la heurística h(n). Si tras estas dos comparaciones persistiera un empate, se dará prioridad a los nodos más antiguos de la lista de abiertos. Téngase en cuenta que si los vecinos de un nodo empatan en f(n) y h(n), se usará el orden anterior (derecha, arriba, izquierda, abajo) para introducirlos en abiertos.

- **Restricción de uso de MetricsProvider**. Se recogerá el valor disponible en la clase MetricsProvider tras la finalización de la ejecución del juego (runOneGame). Muy importante: cualquier otro método para proporcionar las métricas adicionales no será considerado como válido, aunque los resultados sean perfectos.

- **Restricción de implementación**. Se deben respetar de forma estricta los algoritmos planteados en base a las explicaciones y material proporcionado por el equipo docente en las presentaciones de las prácticas. En caso de que una versión “alternativa” de un algoritmo proporcione un camino distinto al esperado, la calificación de dicho mapa será de 0 puntos.

- **Restricción de agrupamiento de acciones.** Se deben implementar los estados de tal forma que cada acción/tick de GVGAI se corresponda a un estado y el algoritmo de búsqueda pase por cada uno de los estados presentes en el camino encontrado. Es decir, no es válido agrupar múltiples estados (por ejemplo, agrupar todos los estados asociados a un viaje completo con la catapulta, considerándolos una única acción, es una representación inválida). En términos prácticos, esto se traduce en que, al generar vecinos, hay que generar estados que siempre tengan coste 1. No sería válido tener estados con coste 2, 3, etc.

- **Restricción relativo a la puerta de salida.** Las puertas cerradas tienen que ser consideradas como obstáculos. Si la puerta está cerrada, esta es un obstáculo que

tiene el mismo comportamiento que la casilla de tipo muro/planta. Es decir, no puede ser atravesada si está cerrada y, es más, si estamos volando, es el otro tipo de casilla (aparte del muro/planta) que nos puede servir de casilla de colisión para transformarnos de vuelta (en la casilla previa a la colisión) de la forma de vuelo/murciélago a la forma de vampiro.

Los algoritmos puntuarán de la siguiente manera:

- Resultados de Búsqueda en Profundidad: 1.5 puntos
- Resultados de A*: 2 puntos
- Resultados de RTA*: 1.5 puntos
- Resultados de LRTA*(k): 2 puntos

El **código** debe estar adecuadamente presentado y **comentado**, explicando los distintos aspectos de la solución propuesta por el estudiante de una forma clara: qué se hace, cómo se hace y por qué. **Un código insuficientemente comentado o con comentarios extremadamente deficientes puede invalidar total o parcialmente el ejercicio al que afecte.**

Es importante remarcar que el profesorado se reserva el derecho a solicitar una defensa oral del trabajo realizado. Se estima que un porcentaje inferior al 20% del alumnado realice dicha defensa oral.

## 4. Memoria y evaluación de cuestiones teórico-prácticas

Junto a la implementación de los distintos agentes deliberativos, se solicita al alumnado que rellene una memoria en la que responderá una serie de cuestiones teórico-prácticas en relación al trabajo desarrollado. Dicha memoria no debe tener ningún tipo de portada ni índice, aunque sí debe tener los elementos identificativos del estudiante (nombre y apellidos) en la cabecera del documento. **Será entregada en formato .pdf y no podrá tener una extensión superior a 5 páginas. Cada página adicional (incluso si es una portada o un índice) restará 0.25 puntos de la nota asociada a la memoria.**

En esta memoria debe desarrollar la **respuesta justificada** de las siguientes preguntas (una pobre presentación o justificación de las preguntas planteadas en la memoria podrá afectar negativamente a las puntuaciones obtenidas en las mismas):

1) Desarrolle una tabla en la que muestre el tiempo de ejecución de la búsqueda [ms], el tamaño de la ruta y todas las métricas específicas por algoritmo de acuerdo con la sección 3 para los mapas 0, 5 y 6 del juego planteado. Indique de forma justificada bajo qué circunstancias cree que es más adecuado utilizar cada uno de los algoritmos implementados en la práctica, es decir, qué pros y contras tiene cada uno, y en qué casos es más recomendable su uso. **[0.5 puntos]**

2) La búsqueda bidireccional (Capítulo 3.5.6 del libro de Russell&Norvig, 4ª edición) es una variación sobre algoritmos de búsqueda en espacio de estados que intenta optimizar el proceso de exploración realizando dos búsquedas simultáneas: una hacia adelante (desde el estado inicial hacia el portal de salida) y otra hacia atrás (desde el estado objetivo hacia la posición inicial). Sin embargo, su eficiencia y viabilidad se ve comprometida en juegos complejos donde el estado de estos está conformado por múltiples elementos dinámicos. Tomemos como ejemplo el juego del curso pasado (véase Figura 5). Se trata de un videojuego sencillo cuyo objetivo consiste en alcanzar un portal de salida recorriendo un mundo laberíntico. Para ello, en primer lugar, el avatar debía evitar las trampas presentes en el mapa (tocarlas supondría la muerte inmediata), y era necesario capturar/recolectar una serie de capas para superar ciertos muros. Más concretamente, había tres tipos de muros: grises (totalmente infranqueables), azul-verdosos (que necesitaban que el avatar portase la capa azul para poder traspasarlos), y marrones (que necesitaban que el avatar portase la capa roja para poder traspasarlos). El avatar solo podía disponer de una única capa en cada instante: partía de un vestuario estándar (sin capa) y, a medida que iba avanzando, podía ponerse las capas que encontrase (para ello, sencillamente bastaba con pasar por la casilla en donde estuviese la capa, y esta desaparecía del mapa). El ponerse una capa implicaba perder la otra (que desaparecía y no podía volver a ser reutilizada), y podía haber más de una capa de cada tipo en el mapa. Es decir, para cruzar la meta, el agente debía de atravesar obstáculos de diferentes colores y, para ello, tomar la capa del mismo color para atravesarlos.

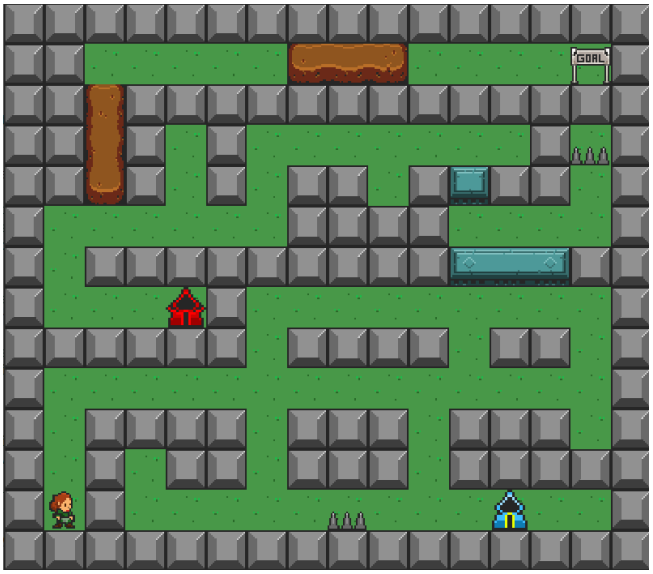


*Figura 5: Ejemplo de un mapa del juego utilizado el curso pasado (Labyrinth Dual), en donde se muestran muros marrones (que requieren la capa roja para ser transitados), muros azul-verdosos (que requieren la capa azul para ser transitados), trampas, puerta objetivo y avatar (en la esquina inferior izquierda).*

En este juego concreto, ¿cómo se plantearía la representación del estado? ¿Qué posibles problemáticas plantearía la búsqueda bidireccional en este juego? Por ejemplo, ¿se puede emplear la misma representación de los estados en la parte "forward" y en la parte "backward"? ¿Qué ocurre cuando se encuentran ambas fronteras? ¿Es el camino encontrado óptimo? **[0.75 puntos]**

3) Una de las características más interesantes del LRTA*(k) es su capacidad de aprender de una ejecución a otra mediante la actualización de los valores de la función heurística. Responda de forma **justificada y teniendo en cuenta todas las posibles casuísticas** qué ocurriría si, tras guardar los valores heurísticos de una ejecución a otra: a) el agente empieza en una posición diferente, b) el estado objetivo es diferente. **[0.75 puntos]**

4) En la resolución de problemas mediante búsqueda, el diseño del estado es uno de los elementos clave. Dicho estado debe contener toda la información necesaria para que el agente tome las decisiones correctas. Si simplificamos el estado sin tener en cuenta sus consecuencias, el algoritmo puede encontrar soluciones que no son válidas en la ejecución real o que no sean óptimas. Así pues, en este apartado se solicita al alumnado que seleccione un elemento del estado distinto a la posición (x, y) y desarrolle un mapa en el que la ausencia de ese elemento en el estado dé lugar a dos caminos hacia la meta diferentes. Deje una captura de pantalla de dicho mapa junto a su justificación y entregue el mapa en la raíz de su entrega como "mapaejercicio4.txt". **[1 punto]**

## 5. Material a entregar

El material a entregar será un fichero ZIP con el siguiente contenido:

- La carpeta del paquete Java, que tendrá el mismo nombre que el definido anteriormente (src_APELLIDO1_APELLIDO2_NOMBRE). Dentro de esta carpeta estarán todas las clases Java correspondientes a los cuatro agentes deliberativos (y otras clases auxiliares usadas por éstos, en caso de ser necesarias), con las siguientes consideraciones:

  a) El **código** debe estar adecuadamente presentado y **comentado**, explicando los distintos aspectos de la solución propuesta por el estudiante de una forma clara: qué se hace, cómo se hace y por qué. **Un código insuficientemente comentado o con comentarios extremadamente deficientes puede invalidar total o parcialmente el ejercicio al que afecte**.

b) Todas las métricas complementarias se recogerán a través de la clase **MetricsProvider**. El contenido mostrado a través de la salida estándar por pantalla será completamente ignorado durante la corrección.

c) No se debe entregar ni el archivo de Test.java ni ningún archivo adicional de la plantilla. A nivel de código se debe entregar exclusivamente el código desarrollado en el paquete de evaluación del alumno y esto debe ser autosuficiente para la ejecución de los algoritmos.

d) Todos los códigos serán ejecutados y corregidos de forma automatizada en base a los criterios expuestos en este documento utilizando Java JDK 25 en un sistema de sobremesa con un Intel(R) Core(TM) i7-14700 y 64 GB de RAM.

- El fichero PDF de la memoria, con una **extensión máxima de 5 páginas**[4], donde se incluyan los **datos del estudiante** en la cabecera, y las **respuestas razonadas** a las cuestiones planteadas.
- En caso de uso de IA generativa, se debe entregar un documento PDF adicional, en el que se incluyan los *prompts* y respuestas proporcionadas por el uso de esta herramienta.

**MUY IMPORTANTE: Sobre el uso de IA generativa y autoría del trabajo**

El uso de herramientas de IA generativa (como ChatGPT, Gemini u otras) se permite exclusivamente como un recurso de apoyo y consulta. Bajo ninguna circunstancia estas herramientas pueden utilizarse como sustituto del razonamiento, el diseño o el trabajo propio del alumno. Es decir, **no se permite el uso de la IA generativa directamente para implementar el código o resolver las cuestiones teóricas planteadas**, pero sí que se puede utilizar para tareas de apoyo (detectar y explicar errores en el código, consultar explicaciones de algoritmos, buscar bibliografía, etc.), **siempre y cuando quede documentado en el documento PDF adicional exigido para ello**.
Se recuerda que el trabajo debe ser **estrictamente individual y debe adherirse a los principios de mérito individual expuestos en la normativa de evaluación y calificación de los estudiantes de la UGR (https://www.ugr.es/sites/default/files/2017-09/examenes.pdf).**

**Incumplir cualquiera de estas normas** implicará el suspenso automático de la asignatura en la convocatoria ordinaria con una **calificación de 0 puntos.**

[4] Esto representa un límite absoluto que incluye todo (desde una posible portada, a un posible índice, a una posible sección bibliográfica final, etc.).